

RAG Systems for Regulatory and Technical Documentation

Architectural Decision Report and Proposal for a MCP-Based Local RAG Server

RAG Research Team
Universidad Nacional de Colombia

August 18, 2026

Abstract

This report is a strategic evaluation framework for deciding what kind of retrieval-augmented generation (RAG) architecture to build for regulatory and technical documentation. It reviews the state of the art as of August 2026 – native multimodal document parsing, hybrid retrieval combined with knowledge graphs, and the emergence of the Model Context Protocol (MCP) as the standard for context servers – and compares two architectural paradigms (custom RAG pipeline versus MCP-based RAG server) and two infrastructure paradigms (local versus API/cloud deployment). Following this evaluation, the report proposes a **MCP-based local RAG server** as the baseline architecture for a controlled experimental comparison between local and API-based implementations, and details the benchmarking methodology that this architecture enables.

1 Introduction

1.1 Purpose and Scope

The purpose of this document is to provide the decision criteria needed to choose the RAG architecture for a knowledge system built around complex documents: regulations, technical documentation, hierarchies, and design diagrams. It is explicitly *not* an implementation guide. It evaluates current industry philosophies (state of the art, 2026), the available tool ecosystem, the complexity level of each alternative, and the user profile for which each alternative is the best fit.

The report is organized in two parts. The first part (Sections 2 to 5) synthesizes the architectural decision guide. The second part (Sections 6 to 9) develops the selected solution: a local RAG system exposed as an MCP server, together with the experimental methodology proposed to benchmark it.

2 State of the Art and Industry Trends (2026)

Extracting knowledge from complex PDFs (regulations, hierarchies, design diagrams) has moved beyond simple text chunking. Three trends define the current landscape.

2.1 Document Processing (SOTA Parsing)

The industry has shifted toward **native multimodal models and semantic parsers** (e.g., LlamaParse, Docling [4]) that understand tables and complex structures without losing formatting. Parsing quality is now treated as a first-class stage of the RAG pipeline rather than an incidental step.

2.2 Hybrid Retrieval and GraphRAG

Vector similarity alone is no longer the standard. Current solutions **combine keyword search, dense vectors, and knowledge graphs** [10] to connect regulatory articles that reference each other, so that retrieval can follow both semantic proximity and explicit document relations.

2.3 Context Standardization

The emergence of the **Model Context Protocol (MCP)** [1, 2] has changed the paradigm: instead of building monolithic RAG applications, the trend is to create *context servers* connectable to any AI client or agent. This report adopts MCP as the interoperability layer of the proposed solution.

2.4 Evaluation Landscape

Alongside these trends, the evaluation tooling has matured: RAG-specific frameworks such as RAGAs [5] and RAGChecker [6], embedding benchmarks (MTEB [8]), vector database benchmarks [9], local inference benchmarks [11], and general RAG evaluation surveys [3] provide the methodological basis for the experimental design in Section 7. For visual-document retrieval, ColPali [7] represents the current research direction, though it is out of scope for the initial prototype.

3 Architecture Paradigm: Custom RAG vs. MCP Server

This section evaluates how to structure the solution at the software level.

3.1 Custom RAG (Ad-Hoc Pipeline)

A custom RAG builds the entire flow from scratch: ingestion, vector database, retrieval, LLM orchestration, and frontend. It offers absolute control and extreme optimization for highly specific workflows, but carries high maintenance cost, a slow development cycle (months), and the risk of “reinventing the wheel” for integrations. It is the right choice for AI-core products that need their own UI, highly specific re-ranking logic, or deep integration with non-standard legacy systems.

3.2 RAG Server via MCP

An MCP-based RAG server packages the retrieval system (documents and regulations) as a server that exposes its capabilities under the MCP standard, allowing any MCP-compatible AI client to consume it. The philosophy is “write the context once, consume it anywhere”. Orchestration and frontend are delegated to existing AI clients, which makes deployment extremely fast (days to weeks) at the cost of functional dependency on the client tool and less control over the final prompt and UI.

Table 1 summarizes both paradigms.

4 Infrastructure Paradigm: Local vs. API (Cloud)

This section evaluates where and how the AI models are processed and where the data is stored.

Attribute	Custom RAG (Ad-Hoc Pipeline)	RAG Server via MCP
Philosophy	Absolute control and extreme optimization for highly specific workflows.	“Write the context once, consume it anywhere.” Interoperability and open ecosystem.
Complexity	High. Requires constant pipeline maintenance and infrastructure management.	Medium/Low. Orchestration and frontend are delegated to existing AI clients.
Target user	Companies with AI-core products that need their own user interfaces, highly specific re-ranking logic, or deep integration with non-standard legacy systems.	Internal teams, design studios, or law firms that already use third-party AI assistants and only need to connect their private data without building a new application from scratch.
Advantages	Total flexibility, frontend customization, millisecond-level latency optimization.	Ultra-fast deployment (days/weeks), zero effort building user interfaces, native integration with developer and analyst workflows.
Disadvantages	High technical debt, slow development cycle (months), “reinventing the wheel” for integrations.	Functional dependency on the client tool (if the AI client reasons poorly, the RAG will fail); less control over the final prompt or the UI.
Ready-made tools	LangChain, LlamaIndex, Haystack, Qdrant/Pinecone, Streamlit/Vercel (for UI).	Official MCP SDKs, Smithery.ai (server registry).
Compatibility	Closed. Only communicates with the systems the development team explicitly programs.	Open and universal with SOTA clients: Claude Desktop, Cursor, Windsurf, Zed, and any MCP-compatible AI agent.

Table 1: Architectural paradigms: custom RAG pipeline versus RAG server via MCP.

4.1 Local Deployment (On-Premise / Edge)

Local deployment runs the embedding and generation models on company-owned hardware. It is the target choice for government entities, law firms, defense, or healthcare organizations with strict privacy regulations (HIPAA, strict GDPR). It guarantees absolute privacy, fixed long-term cost, and immunity to internet outages and provider rate limits. Its weaknesses are the VRAM bottleneck when scaling to concurrent users and the lower reasoning capacity of local models (7B–70B parameters) on complex legal or regulatory jargon. Typical tools are Ollama, vLLM, LM Studio, llama.cpp, and HuggingFace TEI.

4.2 API / Cloud (SaaS)

API-based deployment consumes foundation models hosted by third parties (OpenAI, Anthropic, Google, etc.). It is the target choice for startups, agile companies, and teams that prioritize response quality and delivery speed. It provides state-of-the-art reasoning, near-infinite scalability, and optimized time-to-first-token latency. Its risks are cost leaks on long documents, provider dependency, and legal or compliance barriers when regulation forbids sending data to public clouds. Typical tools are the OpenAI API, Anthropic API, Google Vertex AI, and AWS Bedrock.

Table 2 summarizes both paradigms.

5 Decision-Making Summary

Table 3 summarizes the recommended architecture and infrastructure per business scenario.

Attribute	Local Deployment (On-Premise / Edge)	API / Cloud (SaaS)
Target user	Government entities, law firms, defense, or healthcare companies with strict privacy regulations (HIPAA, strict GDPR).	Startups, agile companies, and teams that prioritize response quality and delivery speed over data concerns (provided B2B no-training agreements are signed).
Complexity	Very High. Requires a DevOps/MLOps profile and costly hardware.	Low. The provider manages the hardware.
Advantages	- Absolute privacy: regulatory data and design secrets never leave the machine. - Fixed long-term cost (no per-token payment). - Immunity to internet outages and provider rate limits.	- Absolute state-of-the-art reasoning (2026 reasoning models, huge context windows). - Infinite scalability: supports spikes of 1,000 users without extra configuration. - Highly optimized time-to-first-token (TTFT) latency.
Disadvantages / where it breaks	- VRAM is the bottleneck: scaling to multiple concurrent users quickly brings the machine down. - Less capable models: local LLMs (7B–70B parameters) often fail to match the deep reasoning required for complex legal/regulatory jargon compared with cloud models.	- Cost leaks: if the regulations are very long and the RAG is inefficient, the bill for millions of tokens can skyrocket. - Provider dependency: global API outages affect the service. - Legal/compliance barriers if regulation explicitly forbids sending data to public clouds.
Ready-made tools	Ollama, vLLM, LM Studio, llama.cpp, HuggingFace TEL.	OpenAI API, Anthropic API, Google Vertex AI, AWS Bedrock.

Table 2: Infrastructure paradigms: local versus API/cloud deployment.

Business Scenario	Architectural Recommendation	Infrastructure Recommendation
Internal legal audit: a team reviews confidential regulations that cannot leave the network by law. They already use local assistants.	MCP Server RAG (to connect to their existing assistants).	Local (Ollama/vLLM) to guarantee total privacy.
B2B SaaS product: they want to sell their own web platform to architects for consulting building regulations.	Custom RAG (they need their own UI, analytics, and flow control).	API / Cloud to scale users quickly and use SOTA models.
Developer/designer productivity: the team needs technical design clarifications directly from their IDE (e.g., Cursor).	MCP Server RAG (integrates natively into their daily work environment).	API / Cloud (assuming the data is not government top-secret).

Table 3: Decision-making summary per business scenario.

6 Proposed Solution: MCP-Based Local RAG Server

6.1 Overview

The selected solution uses a **Model Context Protocol (MCP) server** as the standard interface between an LLM-based client and a fully local Retrieval-Augmented Generation system. The key idea is that MCP is not the RAG system itself: **MCP provides a standardized tool interface through which an LLM can access the RAG system.**

The architecture separates three concerns:

1. **Knowledge layer** — documents, embeddings, and vector storage.
2. **Retrieval layer** — semantic search over the local knowledge base.
3. **Interaction layer** — an MCP server exposing retrieval capabilities as tools to compatible clients.

This separation makes the solution particularly suitable for an experimental comparison between **local execution** and **API-based approaches**.

6.2 Interaction Flow

Conceptually, the MCP server acts as a controlled bridge between an LLM client and the local knowledge base. Figure 1 shows the typical flow.

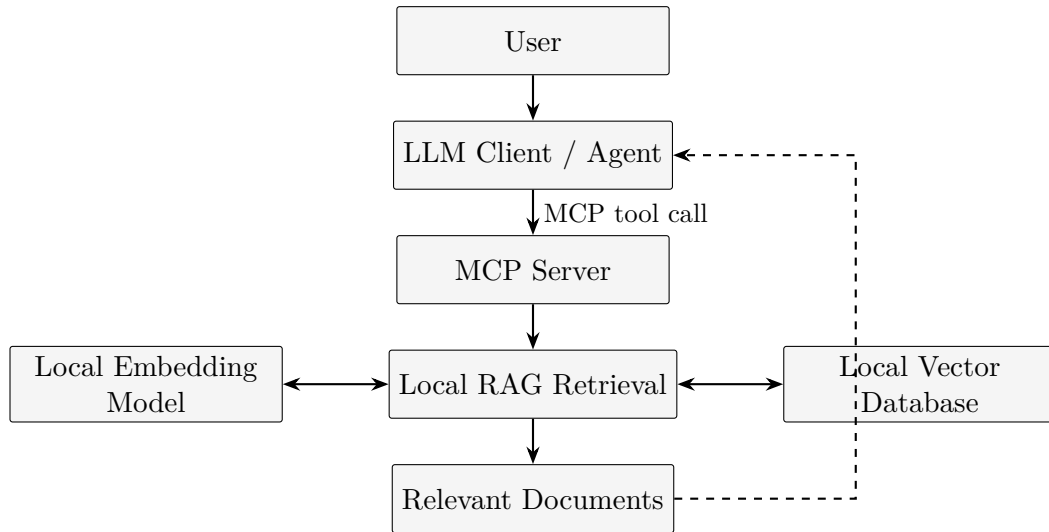


Figure 1: Typical interaction flow between the user, the LLM client, the MCP server, and the local RAG retrieval system.

The MCP server does not need to contain the LLM itself. Its responsibility is to expose well-defined capabilities, such as:

- Searching the local knowledge base.
- Retrieving relevant document fragments.
- Potentially listing available documents.
- Potentially retrieving metadata or document information.
- Providing a consistent interface for future RAG operations.

The LLM decides **when** to use the retrieval tool, while the MCP server controls **how** the retrieval operation is performed.

6.3 Why MCP Is a Good Architectural Choice

Without MCP, an application needs a custom interface between the LLM and the RAG system. With MCP, the RAG capability is exposed through a **standard protocol instead of a client-specific integration**. This reduces the amount of interface code that has to be maintained and makes the RAG server easier to reuse with different MCP-compatible clients.

6.4 Methodological Rationale

The strongest reason to use MCP in this project is methodological rather than purely technical: the objective is to evaluate different ways of providing an LLM with access to knowledge while keeping the underlying retrieval capability as consistent as possible.

An MCP-based architecture establishes a clear experimental boundary (Figure 2), so the client and the server can be treated as separate experimental components.

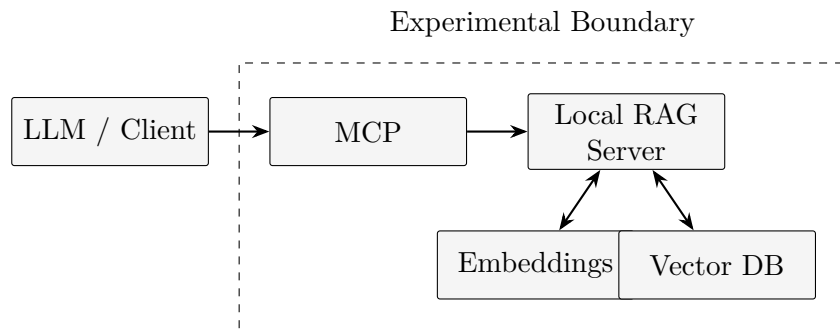


Figure 2: The MCP interface defines the experimental boundary between the LLM client and the local RAG implementation.

This design makes it easier to compare variables such as: local versus API-based embeddings; local versus cloud-based retrieval infrastructure; retrieval latency; end-to-end response latency; CPU utilization; RAM consumption; storage requirements; network dependency; retrieval quality; and operational complexity.

Most importantly, the **retrieval interface can remain stable while the implementation behind it changes**. The same MCP tool (`search_knowledge_base(query)`) can remain in place while the underlying implementation changes from *local embeddings + local vector database* to *API embeddings + remote vector database*. This makes the comparison cleaner because the interface exposed to the LLM does not change.

6.5 Advantages

- **Separation of concerns.** The LLM does not need to know which embedding model, vector database, or storage layout is used, or how similarity search and indexing are implemented.
- **Reusability.** A single MCP server can be consumed by multiple compatible clients (CLI agent, desktop client, other MCP clients) without rewriting the RAG implementation.
- **Local execution.** Documents, embedding model, vector database, retrieval logic, and the MCP server can all stay on the user’s machine, minimizing external dependencies and network traffic.
- **Easy substitution of components.** Embedding models (e.g., all-MiniLM-L6-v2, other local models, API-based models) and vector stores (ChromaDB, other local databases, remote databases) can be replaced independently while the MCP interface remains approximately the same.

- **Good fit for benchmarking.** The server provides a defined boundary from which the RAG subsystem can be measured independently (embedding time, indexing time, retrieval latency, memory and CPU usage, storage, retrieval relevance).
- **Low implementation complexity.** A basic prototype only requires: initialize the MCP server, initialize the local embedding model, initialize the vector database, load/index documents, expose the search operation as an MCP tool, and wait for client requests. No custom protocol, API gateway, or client-specific integration is needed.

6.6 Disadvantages and Limitations

- **MCP adds an integration layer.** If the only requirement is a simple local application performing retrieval, a direct function call is simpler. MCP should be justified by interoperability, tool-based LLM interaction, or architectural separation.
- **Client compatibility.** The benefits depend on the availability of MCP-compatible clients; a conventional application cannot consume an MCP server without an adapter.
- **Process and communication overhead.** A separate MCP process communicating over standard input/output adds overhead relative to a direct function call, which should be considered in precise latency measurements.
- **MCP does not solve RAG quality.** It standardizes access to tools but does not guarantee good chunking, embeddings, retrieval quality, ranking, parsing, citations, or generated answers.
- **Local does not mean resource-free.** Indexing can still consume significant CPU, RAM, disk space, and processing time; model selection and indexing strategy are experimental variables for larger collections.

7 Proposed Experimental Methodology

For a meaningful comparison, the system must use the **same document collection and evaluation questions** across the different implementations (local and API-based).

7.1 Phase 1 — Dataset Preparation

Prepare a controlled, *versioned* document collection containing representative material: PDF documents, technical documentation, plain-text files, source-code repositories, markdown documentation, and standards or regulatory documents. For the Colombian regulatory domain, the ANE/MinTIC compilation [12] is a concrete candidate corpus. Versioning guarantees every experiment uses the same source material.

7.2 Phase 2 — Local Indexing

Process the documents with a reproducible pipeline: **text extraction** → **chunking** → **embedding generation** → **vector index**. Record the number of documents, number of chunks, total text size, indexing time, embedding generation time, and storage size. Document ingestion and retrieval must be treated as separate stages: ingestion creates the searchable knowledge base once, and the MCP tool performs retrieval at query time.

7.3 Phase 3 — MCP Exposure

Expose retrieval through a small number of well-defined MCP tools, e.g., `search_knowledge_base(query, top_k)`. The tool should return enough information for the LLM to understand the retrieved context, ideally including document identifiers or metadata.

7.4 Phase 4 — Controlled Queries

Create a fixed benchmark set of questions. The same questions must be executed against the local RAG, the API-based RAG, and any other implementation under evaluation, without changing the questions between experiments.

7.5 Phase 5 — Performance Measurement

Measure at least the metrics in Table 4. If the goal is specifically to compare local versus API-based architectures, network latency and network dependency must be explicitly measured rather than assumed.

Metric	Purpose
Retrieval latency	Measures local search performance.
Embedding latency	Measures query vector generation.
End-to-end latency	Measures user-perceived performance.
RAM usage	Measures local resource requirements.
CPU usage	Measures computational cost.
Storage usage	Measures infrastructure footprint.
Network traffic	Measures external dependency.
Retrieval quality	Measures relevance of retrieved context.

Table 4: Minimum metrics for the performance measurement phase.

7.6 Comparison Design

The value of this architecture is **decoupling**: instead of comparing entire applications, the experiment compares implementations behind a common tool interface (Figure 3). This reduces the number of variables that change simultaneously and yields a more defensible experimental methodology.

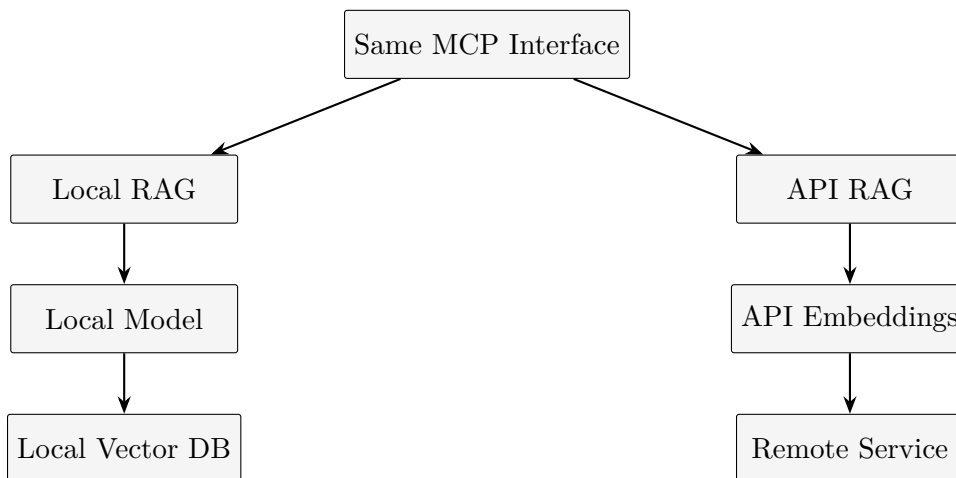


Figure 3: Local and API implementations expose equivalent retrieval capabilities behind the same MCP interface.

The research question becomes: *how does a fully local RAG implementation perform compared with an API-dependent implementation when both expose equivalent retrieval capabilities to an LLM?*

8 Recommended Baseline Architecture

For the initial proof of concept, the recommended architecture is shown in Figure 4. It should be considered a **baseline experimental system**, not necessarily the final production architecture.

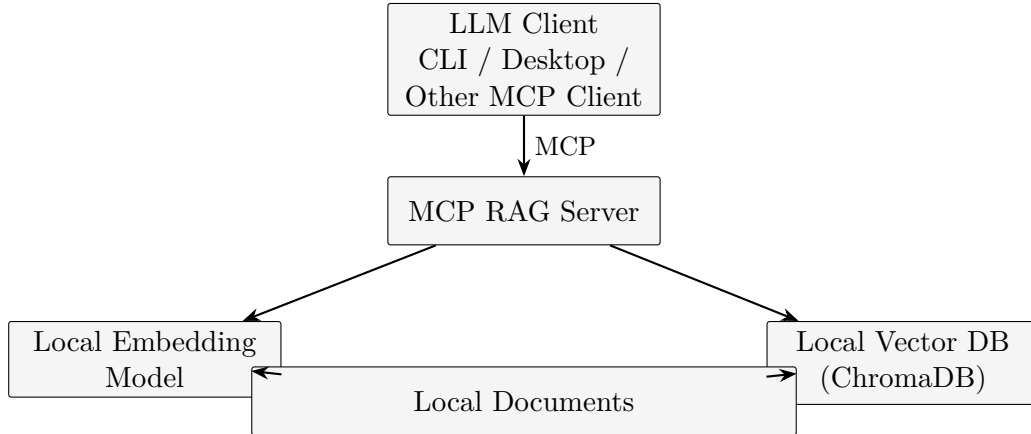


Figure 4: Recommended baseline architecture: MCP-based local RAG server.

Its primary strengths are: clear separation of components; minimal implementation complexity; local execution; a reusable tool interface; compatibility with MCP-based clients; easy substitution of RAG components; and strong suitability for controlled benchmarking.

9 Conclusions: Three Ways to Implement a Local RAG

The decision guide in Sections 3 and 4 converges on three viable ways to build a RAG system for regulatory and technical documentation. They differ mainly in the trade-off between **control** and **time-to-deployment**: the more ready-made the solution, the faster it runs, but the less control it leaves over retrieval quality. Each option is evaluated below in terms of hardware needed, advantages, disadvantages, and target public.

9.1 Option 1: Custom RAG + Local LLM or API

The entire pipeline (ingestion, chunking, embeddings, vector database, retrieval orchestration, and frontend) is built from scratch. Generation runs on a local LLM or on a cloud API, according to the infrastructure decision in Section 4.

Hardware needed. Local inference requires a GPU (8–24 GB of VRAM or more for models in the 7B–70B range) plus CPU, RAM, and disk for indexing and the vector database. The API variant needs no GPU: only a workstation with network access. In practice the 7B–70B range spans very different tiers: 7B–8B models run comfortably on a single consumer GPU with 6–8 GB of VRAM, while 70B models need roughly 22 GB of VRAM even at 4-bit quantization and remain slow (single-digit tokens/second) on one consumer card; usable 70B throughput typically requires 40 GB or more of VRAM across multiple data-center-class GPUs [11].

Advantages. Total control over every stage of the pipeline; custom user interface, analytics, and re-ranking; millisecond-level latency optimization; no dependency on third-party clients. The API variant also provides state-of-the-art reasoning.

Disadvantages. High technical debt; development cycles of months; constant pipeline maintenance; “reinventing the wheel” for integrations. The local variant is constrained by VRAM and by the reasoning capacity of local models on complex legal or regulatory jargon.

Target public. Companies with AI-core products (e.g., B2B SaaS) and teams that need their own frontend, analytics, or deep integration with non-standard legacy systems.

9.2 Option 2: Open-Source MCP RAG Software (Ready to Use)

Existing ready-made MCP RAG servers package a local RAG system behind the standard MCP interface with zero code. A representative example is `mcp-local-rag` [13]: a local-first server with hybrid keyword (BM25) + semantic search, launched with a single `npm` command, configured by pointing a `BASE_DIR` at the documents, and consumable by any MCP client (Cursor, Claude Code, Codex, etc.).

Hardware needed. Minimal. Embeddings run on CPU by default (all-MiniLM-L6-v2 on ONNX Runtime), storage is a file-based vector database (LanceDB), and after the initial model download, ingestion and search work fully offline. No API keys, Docker, Python, or external database are required.

Advantages. Deployment in minutes or hours; no development effort; standard MCP interface reusable across clients; full local privacy; hybrid search quality out of the box.

Disadvantages. Limited control over chunking, embeddings, and retrieval internals; retrieval quality is bounded by the tool’s features and default models; customization requires extending or forking third-party code; roadmap and maintenance are owned by an external project.

Target public. Internal teams, law firms, and individual developers who need fast, private document search integrated with their existing AI clients, and teams that want to validate a corpus before committing to a custom build.

9.3 Option 3: Custom RAG Server via MCP with the Python SDK

A custom MCP server is built with the official Python SDK [2], wrapping a local RAG pipeline (local embeddings + local vector database) and exposing retrieval as MCP tools. This is the solution proposed and detailed in Section 6.

Hardware needed. Same profile as the local path of Option 1 minus the LLM: embeddings run on CPU or optionally on GPU, and the index needs RAM and disk. No GPU is strictly required for a prototype.

Advantages. Full control of retrieval quality behind a standardized interface; separation of concerns; components (embedding model, vector database) are independently substitutable; the server is benchmarkable as an independent subsystem; reusable across any MCP-compatible client.

Disadvantages. Development and maintenance effort (small for a prototype but real); MCP adds a process and communication layer; benefits depend on the client tool’s reasoning quality.

Target public. Research teams running controlled local-versus-API experiments, developers who need to connect an LLM to internal documentation, and organizations with sensitive documents that require control over the retrieval pipeline.

9.4 Comparison and Bottom Line

Table 5 summarizes the three options.

Table 5: Comparison of the three implementation options.

Attribute	Option 1: Custom RAG + Local LLM or API	Option 2: Open-Source MCP RAG Software	Option 3: Custom MCP RAG Server (Python SDK)
Hardware needed	- Local: GPU (8–24 GB VRAM+) for the LLM; CPU/RAM/disk for indexing and vector DB. - API: no GPU; workstation + network.	- CPU only (default ONNX Runtime). - File-based vector DB (LanceDB). - Offline after the first model download.	- CPU (optional GPU) for embeddings. - RAM/disk for the index. - No GPU required for a prototype.
Advantages	- Total control of every stage. - Custom UI, analytics, re-ranking. - Millisecond latency optimization. - SOTA reasoning via API.	- Deployment in hours; zero code. - Standard MCP interface. - Fully local and private. - Works with any MCP client.	- Full retrieval control + standard interface. - Substitutable components. - Independently measurable. - Reusable across clients.
Disadvantages	- High technical debt. - Months of development. - Constant maintenance. - VRAM and local model limits.	- No control over chunking, embeddings, retrieval internals. - Quality bounded by the tool. - Roadmap owned by a third party.	- Own development and maintenance. - MCP adds a process/communication layer. - Client-tool dependency.
Target public	- AI-core companies and B2B SaaS. - Teams needing their own frontend and analytics.	- Internal teams, law firms, individual developers. - Quick private search; corpus validation pilots.	- Research teams running local-vs-API experiments. - Developers and organizations with sensitive documents.

There is no single winner: the choice depends on whether the priority is control (Option 1), speed of deployment (Option 2), or a balance of both (Option 3). For this project, **Option 3 is the recommended baseline**: it keeps the retrieval pipeline fully controllable and measurable while the MCP interface provides the stable boundary required by the experimental methodology of Section 7. Option 2 is the ideal pilot to validate the document corpus in hours, and Option 1 remains the path for a future product with its own user interface and analytics.

MCP for interoperability + local embeddings for privacy and independence + local vector storage for retrieval + a controlled benchmark for comparing local and API-based implementations.

References

- [1] Model Context Protocol Specification (2026-07-28). <https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/server/tools.mdx>
- [2] Official MCP Python SDK. The `main` branch README documents v1.x as the current stable release; v2 is pre-alpha and under active development as of August 2026. <https://github.com/modelcontextprotocol/python-sdk>
- [3] RAG Evaluation Survey (2026). SN Computer Science. <https://link.springer.com/article/10.1007/s42979-026-05134-x>
- [4] Docling: Document ingestion and parsing. <https://github.com/docling-project/docling>
- [5] Es, S., James, J., Espinosa-Anke, L., Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. EACL 2024. <https://aclanthology.org/2024.eacl-demo.16/>
- [6] RAGChecker: A Fine-grained Framework for Diagnosing RAG (2024). arXiv:2408.08067. <https://arxiv.org/abs/2408.08067>
- [7] ColPali: Efficient Document Retrieval with Vision Language Models (2024). arXiv:2407.01449. <https://arxiv.org/abs/2407.01449>
- [8] MTEB: Massive Text Embedding Benchmark. https://docs.mteb.org/overview/available_benchmarks/
- [9] Vector Database Benchmark (2026). Journal of Big Data and Artificial Intelligence. <https://jbdai.org/index.php/JBDai/article/view/80>
- [10] Microsoft Research. GraphRAG. <https://www.microsoft.com/en-us/research/project/graphrag/overview/>
- [11] Local Inference Benchmark (July 2026). Vendor-published benchmark (Plugsy); figures are directionally useful but were not produced by an independent or peer-reviewed source. <https://plugsy.com/local-inference-benchmark>
- [12] ANE/MinTIC Regulatory Compilation (Colombia). https://normograma.mintic.gov.co/mintic/compilacion/cndstr_ane_agencia_nacional_espectro.html
- [13] mcp-local-rag: Local-first RAG server for developers (MCP). MIT License. <https://github.com/shinpr/mcp-local-rag>